QualiAdept

Session 1: Web architecture, DOM structure and HTML fundamentals

Chapter 1: Web Architecture - How the Internet and Browsers Work 2
Learning objectives 2
1.1 From Manual QA to Automation: Changing Perspective 3
1.2 Basic Architecture: Client - Server 3
1.3 The life cycle of a request: HTTP Request & Response 4
1.4 Why is this critical for a QA Engineer? 4
1.5 Did you know that...? 4
1.6 Applied Story: “The Digital Restaurant” 5
1.7 Practical Exercises 5
1.8 Answers to Questions & Solutions to Exercises 6
Chapter 2: Introduction to HTML and Semantic Tags 6
Learning objectives 6
2.1 What is HTML? (It is not a programming language!) 6
2.2 Anatomy of an HTML Element: Tags and Content 7
2.3 Basic structure of an HTML document (Boilerplate) 8
2.4 Semantic HTML: Why does it matter to us as testers? 8
2.5 HTML Attributes: The Anchors of Automation 9
2.6 Did you know that...? 10
2.7 Applied Story: “The Foundation and the Bricks of the House” 10
2.8 Practical Exercises 11
2.9 Answers to Questions & Solutions to Exercises 11
Chapter 3: DOM Structure and Practical Workshop (Task Tracker) 13

Learning objectives....13
3.1 What is the DOM (Document Object Model)?....13
3.2 Family Tree: Parents, Children and Siblings....13
3.3 Why is the DOM the “Playground” of Automation?....14
3.4 How does Playwright "read" the DOM? (A look under the hood)....15
3.5 Practical Workshop: Creating the extended skeleton for the "Task Tracker" application....15
3.6 Did you know that...?....18
3.7 Applied Story: “The Web Family Tree”....18
3.8 Practical Exercises....18
3.9 Answers to Questions & Solutions to Exercises....19
Chapter 4: Browser Memory, HTTP Methods, and Session 1 Topic....20
Learning objectives....20
4.1 HTTP Methods: GET vs. POST (What happens to the form data?)....21
4.2 Browser Memory: Cookies, Local Storage and Session Storage....21
4.3 DevTools: Application Tab (QA's Secret Weapon)....22
4.4 Did you know that...?....22
4.5 Applied Story: "The Postcard and the Armored Parcel"....23
4.6 Practical Exercises....23
4.7 Answers to Questions & Solutions to Exercises....23
4.8 Homework (Session 1 Project) & QualiAdept Cloud Evaluation Platform....24
Introduction to the Certify QualiAdept Platform....24
How to Self-Validate (Homework Assignment)....25

Chapter 1: Web Architecture - How the Internet and Browsers Work

Learning objectives

> 🧠 At the end of this chapter, you will be able to:
>
> - Explain the fundamental difference between the user interface (Frontend) and the server

(Backend).

- You understand the concepts of Client, Server and Database.
- You follow the path of an HTTP request from pressing the Enter key to displaying the site.
- You use the "Network" tab in DevTools to intercept your first Request.

1.1 From Manual QA to Automation: Changing Perspective

In manual testing, you interact with the application exactly like an end user: you click buttons, fill out forms, and check if the visual result is correct (Black Box Testing).

As a future **QA Automation Engineer**, you need to go a level deeper. When an automated test fails (e.g. Playwright can't find a button), you need to know *Why* failed. To do that, you need to understand the path the code takes from the server to your screen. We're no longer just testing "what you see," we're testing the infrastructure.

1.2 Basic Architecture: Client - Server

Almost every modern web application (including the "Task Tracker" application we're going to build) runs on the architecture **Client-Server**.

- **The client:** It is the application that requests information. Most of the time, the client is the web **Browser** (Chrome, Firefox, Safari) from your laptop or phone. When we write automation scripts, our Playwright program will act as a "robot client".
- **Server:** It is a powerful computer, connected non-stop to the internet, that "serves" data. This is where the "brain" of the application (the Backend) lives and where the files (HTML, CSS, images) are stored.
- **Database:** It is the "file cabinet" of the server. This is where long-term information is stored (e.g. user accounts, encrypted passwords, saved tasks).

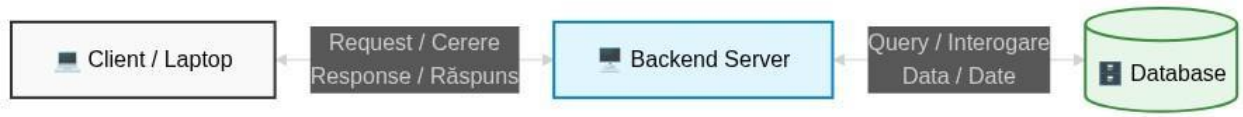

Fig.1 - 3-Tier architecture diagram. The bidirectional arrows indicate synchronous communication between the user interface (Frontend), application logic (Backend), and persistence layer (Database).

1.3 The life cycle of a request: HTTP Request & Response

Communication between Client and Server is done through a set of rules called **HTTP protocol** (HyperText Transfer Protocol). Think of HTTP as the common language that both computers speak.

Here's what happens, step by step, when you type www.emag.ro in your browser and press Enter:

- **HTTP Request:** Your browser (Client) sends a message to the eMAG server. The message says:*"Hello, please give me the home page"*This message is called a GET request.
- **Server Processing:** The server receives the message. It looks in the database to get the latest products, assembles a web page and prepares it for delivery.
- **HTTP Response:** The server sends a "packet" back to the browser. The packet contains:
  - And **Status Code** (ex: 200 OK - everything is perfect, or 404 Not Found - the page does not exist).
  - **HTML code** (page structure).
  - **CSS and JavaScript** (visual design and logic).
- **Rendering (Display):** Your browser receives the packet, reads the code from top to bottom, and "draws" the buttons and images on your screen.

1.4 Why is this critical for a QA Engineer?

If a user clicks "Login" and the screen remains locked, a QA Manual will write a bug:*"The login button doesn't work"*.

A QA Automation that understands the architecture will open **DevTools**, will see that the Request to the server went out, but the Server returned 500 Internal Server Error. The reported bug will be: *"The server returned error 500 when accessing the login endpoint"* The first report is vague; the second helps the developer fix the problem in 5 minutes.

1.5 Did you know that...?

> 💡 **Did you know that...**
>
> - **Lost packets:** Data sent over the internet doesn't travel as one big chunk. It's broken up into thousands of small "packets," which can take different routes around the world. Your browser reassembles them at their destination.
> - **Basic Status Codes:** There is a universal rule for HTTP response codes that every QA

should know:

- 2xx - Success (Everything went well).
- 3xx - Redirect (Page has moved).
- 4xx - Client Error (You made a mistake, e.g. you searched for a URL that does not exist - 404).
- 5xx - Server Error (The server crashed).

1.6 Applied Story: “The Digital Restaurant”

The best analogy for Client-Server architecture is a restaurant.

- **You are the Client (Browser):** You sit at the table and look at the menu (Interface - UI).
- **The waiter is the HTTP Request:** You say to the waiter:*"I want a pizza"* He takes your order and takes it to the kitchen.
- **The Kitchen is the Server (Backend):** The chef takes the order, gathers the ingredients, prepares the pizza, and applies the business rules (e.g., "No onions").
- **The Food Pantry is the Database:** From there the chef takes the raw ingredients.
- **The waiter returns (HTTP Response):** It brings your pizza to the table along with a status ("Enjoy!" = 200 OK, or "We're out of dough" = 404 Not Found).

As an Automation Tester, your job is not just to taste the pizza (UI Testing), but sometimes to intercept the waiter on the way (API Mocking) to see if he wrote down the order correctly.

1.7 Practical Exercises

Exercise 1: Intercepting your own Request (Network Inspector)

We will use the main weapon of any QA engineer: Browser DevTools.

- Open Google Chrome.
- Right-click anywhere on a blank page and choose **"Inspect"** (or press F12 / Ctrl+Shift+I).
- In the top menu of the panel that opened, click on the tab **Network** (Network).
- Type www.wikipedia.org in the browser's address bar and press Enter.
- *Your task:* Look at the cascade of files that appear in the Network tab. Find the first file in the list (usually called wikipedia.org). Click on it and identify in the "Headers" section which is the **Status Code** received.

1.8 Answers to Questions & Solutions to Exercises

Solution Exercise 1:

If you followed the steps correctly, the first item in the Network tab is the Main HTML Document. When you click on it, in the right side panel, under the **General**, you will see the Status Code field. It should be 200 OK (green).

> 📝 *Congratulations! You have just successfully intercepted and analyzed your first Client-Server transaction. This habit will save you countless hours of frustration when debugging our automated tests.*

Chapter 2: Introduction to HTML and Semantic Tags

Learning objectives

> 🧠 At the end of this chapter, you will be able to:
> - Explain the role of HTML in building a web page.
> - Write HTML elements correctly, respecting the opening and closing syntax.
> - You understand the difference and importance of critical attributes for automation (id, class, data-testid).
> - You build a simple, semantically structured web page, ready to be intercepted by a Playwright script.

2.1 What is HTML? (It is not a programming language!)

It is a common confusion at the beginning of the journey. **HTML (HyperText Markup Language)** it is NOT a programming language. You cannot write logic like "if 2+2=4, then display an alert" with it.

HTML is a **markup language,** its sole and exclusive role is to structure information, telling the browser *This* represents each piece of text: "This is a major heading," "This is a paragraph," "Here we have a list," or "This is a clickable button."

As a QA Automation Engineer, HTML is your map. If you don't know how to read the map, your robot (Playwright) will get lost.

2.2 Anatomy of an HTML Element: Tags and Content

To "mark up" text, HTML uses **Tags**, always written between angle brackets < >.

Most HTML elements have a 3-part structure:

- **Opening tag:** Marks the beginning of the element. Ex: <button>
- **Content:** What the user will actually see on the screen. Ex: Send Order
- **Closing tag:** Marks the end of the element. It is distinguished by the addition of a slash / (slash). Ex: </button>

Complete element: <button>Submit Order</button>

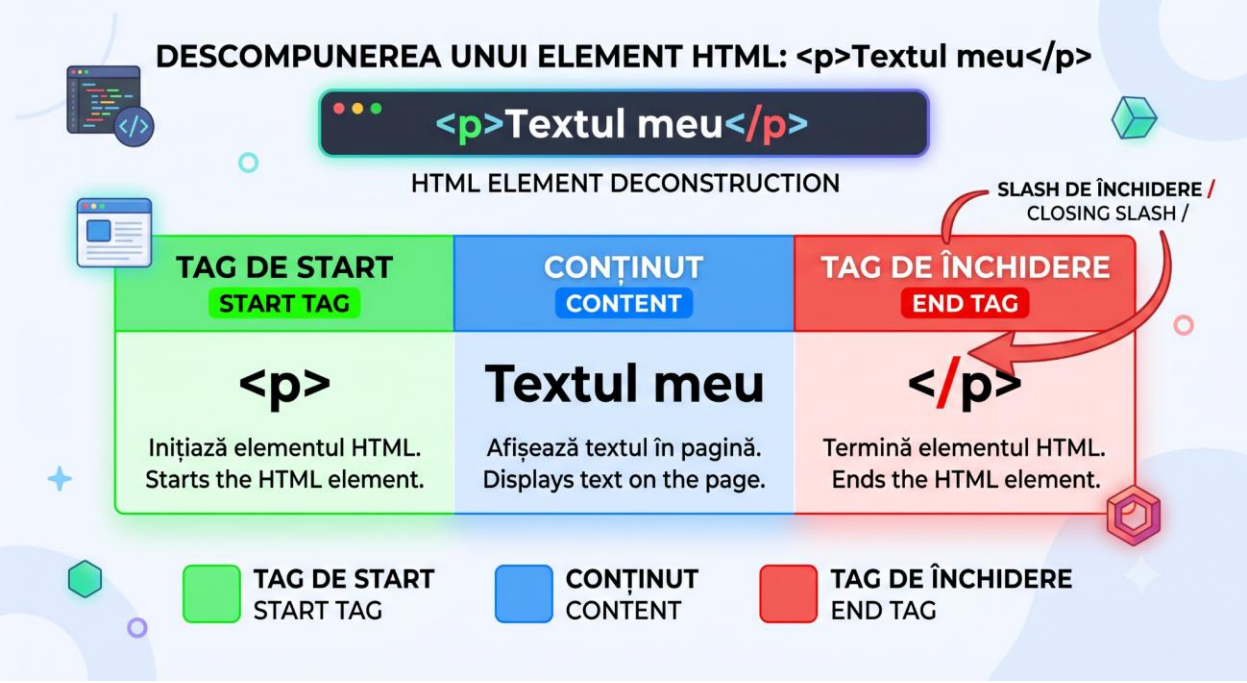

Fig. 2 - Anatomy of an HTML element. Breaking down the <p> tag into the start tag (green), content (blue), and end tag (red), highlighting the slash (/).

Attention (Self-closing elements):

There are certain elements that cannot contain text and therefore do not need a separate closing tag. The most common in QA are images and input fields:

-

- <input type="text" />

2.3 Basic structure of an HTML document (Boilerplate)

Every valid HTML file in the world follows a standard boilerplate. Here's what each area looks like and what it represents:

```
<!DOCTYPE html> <!-- 1. Document type declaration: Tells the browser that we are using HTML5 (the latest version). -->
<htmljust="ro"> <!-- 2. Root Element: All the code is inside this tag. -->

    <head> <!-- 3. The "Brain" Part (Metadata) -->
        <!-- This is information FOR the browser, NOT for the user. Nothing here is visible on the white page (exception: the title in the top tab). -->
        <meta charset="UTF-8">
        <title>My QA App</title>
    </head>

    <body> <!-- 4. The "Visible" Part -->
        <!-- Here you put ABSOLUTELY EVERYTHING you want the user to see on the screen and everything you will test automatically: buttons, forms, texts, images. -->
        <h1>Welcome to Task Tracker!</h1>
        <p>This is where we will add our tasks.</p>
    </body>

</html>
```

2.4 Semantic HTML: Why does it matter to us as testers?

In the 2000s, developers used a generic tag called to divide the page (e.g. ,). It was a hard-to-read mess.

HTML5 introduced **Semantic Tags**. These are tags that clearly describe their role. They are of great help to search engines (SEO), software for the visually impaired, and also to **us (QA Engineers)**, because they make the source code infinitely easier to investigate:

- <header>: The header of the page or section (where the logo is located).
- <nav>: A section containing navigation links (Menu).

- <main>: The main and unique content of the page.
- <section>: A generic thematic section.
- <footer>: The footer of the page (where the Copyright text or contact links are located).

2.5 HTML Attributes: The Anchors of Automation

If the Tags say *what is* an element, **Attributes** offers *Additional Information* about that element. Attributes are written **always in the opening tag**.

The syntax is: attribute_name="value".

This is the most important concept in this chapter for your future role as a QA Automation. These attributes will be the "hooks" that you will hang on to in your Playwright code to find the buttons on the page.

Attribute	**Example**	**Explanation & Relevance for QA**
id	<button id="login-btn">	**CRITIC!** The ID must be unique (never repeated on the same page). It is the fastest, most stable and preferred way to locate an element (ex: page.locator("#login-btn")).
class	<p class="error-text">	Defines a "class" of elements, typically used to color them the same way via CSS. Multiple elements can have the same class. We use it to extract lists of elements.
type	<input type="checkbox">	It tells us what kind of input it is (text, password, check

		mark, radio button). Very useful when we have dozens of inputs.
name	<input name="email">	Commonly used in forms to send data to the server. A great locator in QA.
data-*	<button data-testid="submit-login">	**The Holy Grail of Automation!** (ex: data-testid, data-qa). These are custom attributes. Good developers put them specifically for us, so that our tests don't fail if someone changes the design (CSS classes).

2.6 Did you know that...?

> 💡 Did you know that…
>
> - **HTML is not "Case Sensitive":** The browser doesn't care whether you write <BUTTON>, <Button> or <button>. However, the global standard, strictly recommended in the industry (and which we will use), is to write exclusively in lowercase.
> - **Duplicate IDs break automated tests:** If a developer makes a mistake and puts id="submit" on two different buttons, the Chrome browser is forgiving and will render the page without any visible errors. But your Playwright script looks in the DOM, sees the first element with that ID, clicks on it, and **completely ignores the second one.** A lot of "flaky tests" come from this developer mistake!

2.7 Applied Story: “The Foundation and the Bricks of the House”

If you think of a web page as a house:

- **Boilerplate Structure** (<html>, <head>, <body>) represents the foundation, roof, and exterior walls. You can't have a house without them.
- **HTML tags** (<h1>, <p>, <button>) are the bricks, windows and doors. They say*This*it's there: "Here we put a door."
- **Attributes** (id, class) are the labels you stick on these elements for the control team. If you tell a worker (test script) "Check if the door is closed", it will ask "Which door? There are 10!". If you tell it "Check the element with id='main-entrance-door'", it will go straight to the target, without any confusion.

In automated testing, you will be the inspector who sends robots to check "tags" (attributes) on the construction site.

2.8 Practical Exercises

Exercise 1: Attribute Hunt (DevTools)

Let's use fresh knowledge in the real world.

- Open Google Chrome and go to www.emag.ro (or any large site).
- Right-click on the main search bar and press **"Inspect"** (or Inspect).
- In the Elements panel, the code will light up in the row corresponding to the search bar.
- *Your mission:*Identify and write down somewhere what **id** has that search <input>. But the attribute **type**?

Exercise 2: Write your own Semantic code

Open a text editor (e.g. Notepad or VS Code, without running it in the browser yet) and write a block of HTML code (just what would come inside the <body>), containing:

- A <header> area where you can put a main title <h1> with the text "My Store".
- A <main> area where you can have a form with:
  - Un <input> de tip text, cu id="search-box".
  - A <button> that has the automation-specific attribute data-testid="search-btn" and the text "Search Product".

2.9 Answers to Questions & Solutions to Exercises

Solution Exercise 1:

Although sites can update, usually on eMAG (or similar sites), the search bar will be an <input> tag. You will notice that it has an attribute like type="search" or type="text". The ID is usually very descriptive, like id="searchboxTrigger" or simply id="search". That's how you would locate it in an automated test!

Solution Exercise 2:

Here's what a code written "by the book" looks like, respecting the standards we will rely on in Playwright:

```
<header>
    <h1>My Store</h1>
</header>
<main>
    <input type="text" id="search-box">
    <buttondata-tests="search-btn">Search Product</button>
</main>
```

If you managed to write this snippet correctly, with the angle brackets and quotation marks in place, you're ready to dive deep into the DOM Structure in the next chapter!

Chapter 3: DOM Structure and Practical Workshop (Task Tracker)

Learning objectives

📝 At the end of this chapter, you will be able to:

- Explain what the Document Object Model (DOM) is and how it differs from raw HTML code.
- Identify the family relationships (Parent-Child-Sibling) between web elements on a page.
- You understand exactly how Playwright interacts directly with the DOM structure, bypassing the classic graphical interface.
- You build a complex HTML interface (forms, selections, tables) for the "Task Tracker" web application, adding testability attributes.

3.1 What is the DOM (Document Object Model)?

If HTML is the source code (text) sent by the server, **DOM-ul (Document Object Model)** it is the "live" representation of that code, built by the browser in its memory.

When the browser (Chrome, Edge) receives the .html file, it doesn't display your text directly. It takes that text, parses it (reads it piece by piece), and turns it into an interactive, tree-like data structure. That tree is the DOM.

Once the DOM is created, the browser uses it to "draw" (render) the page on the screen. Later, when we use JavaScript, we can modify the DOM in real time (for example, adding a new element to the screen without reloading the page).

3.2 Family Tree: Parents, Children and Siblings

To be able to locate elements later with Playwright (especially when we don't have useful IDs), we need to understand how elements are "related" in the DOM. The DOM is a strict hierarchy:

- **Root:** It is the starting point, always the <html> element.
- **Parent:** An element that directly contains another element. For example, <body> is the parent of all visible elements on the page.

- **Child:** An element directly inside another element. An <h1> placed inside a <header> is the child of the header.
- **Siblings:** Elements that share exactly the same parent. Two <p> paragraphs below each other in a are siblings.

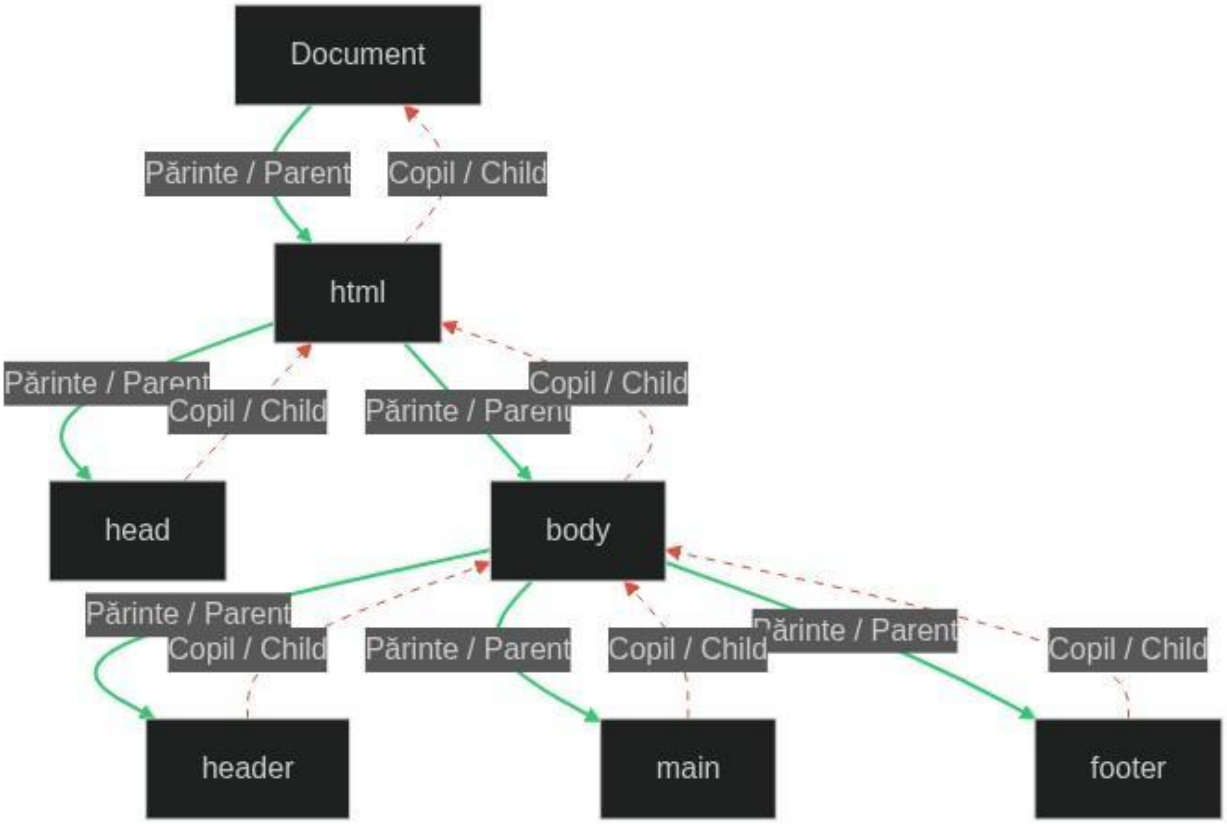

Fig. 3- The hierarchical structure of an HTML document. The Tree Graph diagram exemplifies the Parent-Child relationships between elements. The main node (Document) contains the html element, which is divided into head and body. In turn, body hosts the three visible areas of the page: header, main and footer. The colored labels highlight the subordination links between these nodes.

3.3 Why is the DOM the “Playground” of Automation?

Here it is **The Golden Rule of Automation**: Testing robots (like Playwright or Selenium) **NOT** They have eyes. They don't look at the pixels on your monitor.

When you tell Playwright to "Click the Login button", the script doesn't look for a blue rectangle on the screen. It goes "under the hood", traverses the DOM tree and looks for a "node" (element)

that matches your description (e.g. id="login-btn").

If an element exists in the DOM, but is visually hidden by CSS (e.g. made transparent), Playwright will know it's there, but will politely tell you:*"The element is in the DOM, but it is not visible enough to be clickable"*.

This is why understanding the DOM transforms you from a simple test writer into an automation engineer capable of debugging.

3.4 How does Playwright "read" the DOM? (A look under the hood)

Old testing tools tried to simulate the physical movement of a mouse on the screen. It was a slow and unstable process. Playwright does something much smarter: **connects directly to the browser's "nervous system"**(through something called*Chrome DevTools Protocol*).

- **Speed of light:** Because Playwright queries the DOM tree directly from the browser's RAM, it can find one element out of 10,000 in a few milliseconds.
- **Auto-Waiting:** Playwright doesn't just read the static DOM. It "listens" to DOM events. If you tell it to click a button that hasn't yet appeared on the page, it will sit and "listen" to DOM mutations until the button is "attached" to the structure, then wait for it to become "visible" and "interactive" before performing the click.
- **Penetrating beyond the surface:** Playwright can force interactions. If an element is covered by another transparent div (a common UI bug), you can use element.click({ force: true }), which tells Playwright:*"Ignore what's drawn on the screen, go directly to the button's DOM node and fire the JavaScript Click event on it!"*.

3.5 Practical Workshop: Creating the extended skeleton for the "Task Tracker" application

Enough with the theory! Let's build a **HTML skeleton** much richer for our application **To-Do List** We'll add selection forms and a table, perfect for practicing automation later.

Step by step instructions:

- Open your text editor (Visual Studio Code).
- In the folder **QA_Task_Tracker** previously created, open the index.html file.
- We will replace the old content in **<body>** with a much more complex interface. Notice the

use of the <form>, <select>, and <table> elements, and the special attention paid to the attributes **data tests**!

Code to copy inside <body>:

```
    <header>
        <h1>Task Tracker Pro - QA Edition</h1>
        <p>Complete automation practice platform</p>
    </header>

    <main>
        <!-- Add task section -->
        <section class="task-input-section">
            <h2>Add a New Task</h2>
            <form id="add-task-form">
                <label for="task-name">Task Name:</label>
                <input type="text" id="task-name"data-tests="input-task-
name" placeholder="Ex: Write E2E scripts" required>

                <label for="task-priority">Priority:</label>
                <select id="task-priority"data-tests="select-priority">
                    <option value="low">Low</option>
                    <option value="medium" selected>Medie</option>
                    <option value="high">Criticism</option>
                </select>

                <label for="due-date">Deadline:</label>
                <input type="date" id="due-date"data-tests="input-due-
date">

                <button type="submit" id="add-task-btn"data-tests="submit-
new-task">Save Task</button>
            </form>
        </section>

        <!-- Task display section (List Type) -->
        <section class="task-list-section">
            <h2>Task-uri Active</h2>
            <ul id="active-tasks-list">
                <that class="task-item" data-task-status="pending">
                    <input type="checkbox" class="complete-checkbox"data-
tests="check-task-1">
                    <span>Learn DOM architecture</span>
```

```
                    <span class="badge priority-high">Criticism</span>
                    <button class="delete-btn"data-tests="delete-task-
1">Delete</button>
                </that>
            </ul>
        </section>

        <!-- History Section (Table Type) -->
        <section class="task-history-section">
            <h2>Completed Tasks History</h2>
            <table id="history-table" border="1">
                <thead>
                    <tr>
                        <th>ID</th>
                        <th>Task Name</th>
                        <th>Completion Date</th>
                    </tr>
                </thead>
                <tbody>
                    <tr>
                        <td>#1001</td>
                        <td>HTML Boilerplate Setup</td>
                        <td>09-Aug-2026</td>
                    </tr>
                </tbody>
            </table>
        </section>
    </main>

    <footer>
        <p>(c) 2026 QualiAdept Bootcamp</p>
    </footer>
```

- **Visual testing:** Save the file and refresh Chrome.

What have I achieved?

We created a real challenge for QA! Now we have:

- And **Formulate** (<form>) which can test the native behavior of the browser.
- And **Dropdown** (<select>) which requires special commands in Playwright (locator.selectOption()).
- A selector of **Date** (<input type="date">).

- And **Table** data, perfect for practicing iterating through lists and rows (e.g. finding text in the 2nd column of a table).

3.6 Did you know that...?

💡 **Did you know that...?**

- **The DOM is updated "Live":** Unlike the static index.html file on your hard drive, the DOM can constantly change after the page loads. If you use the "Elements" tab in DevTools, you can see the DOM*current*, which can contain dozens of elements dynamically added by JavaScript, which you wouldn't see if you just right-click -> "View Page Source".
- **The "data-testid" attribute does nothing visually:** A developer can add any attribute they want by inventing the word data- in front of it. data-testid does not change the color or behavior of the element, it is simply a beacon placed by the developer that Playwright sensors automatically look for using page.getByTestId().

3.7 Applied Story: “The Web Family Tree”

🧠 Let's imagine the DOM as a traditional family.

- The supreme grandfather is **The document**.
- Grandpa has two children: **<head>** (the introverted child, the brain of the family, who sits hidden and thinks about metadata) and **<body>** (the extroverted, visible child who interacts with the world).
- In **<body>** The grandchildren live in: Header <header>, Main Area <main> and Footer <footer>. All three of these are Brothers.

If you tell Playwright, "Find me a button," it might exist in both the Form and the Task List. To be precise, you sometimes have to tell it:*"Go to the Father <form>, and inside him find my Child <button>"*This "route" through the DOM family is called*Basic Path (DOM Traversal)*.

3.8 Practical Exercises

Exercise 1: Inspect new elements (DevTools)

- Open your new index.html in Chrome and press F12.
- Inspect the <select id="task-priority"> element.
- Expand it from the arrow.
- *Question:*Notice the <option> tags inside it? Are they the Parents, Children, or Siblings of the <select> tag?

Exercise 2: Add a new functionality to the table

Our History table currently only has one row of data (inside the <tbody>).

Your task: Go to the index.html file and add a second row to the table (a new <tr> tag), with the following data: ID "#1002", Name "Understanding Client-Server Architecture", and Date "Today".

Save and check in browser.

3.9 Answers to Questions & Solutions to Exercises

Solution Exercise 1:

The <option> tags sit directly inside the <select> tag, so they are **Children** of the select tag (and the select tag is their Parent). In Playwright, when you want to extract all the options from a dropdown, you will ask the robot to count all the "children" of that select.

Solution Exercise 2:

Your new inner <tbody> tag should look like this:

```
<tbody>
    <tr>
        <td>#1001</td>
        <td>HTML Boilerplate Setup</td>
        <td>09-Aug-2026</td>
    </tr>
    <!-- The line you added: -->
    <tr>
        <td>#1002</td>
        <td>Understanding Client-Server Architecture</td>
        <td>10-Aug-2026</td>
    </tr>
```

```
            </tbody>
```

If the table expanded correctly in the browser, you are able to independently manipulate a valid HTML structure!

Chapter 4: Browser Memory, HTTP Methods, and Session 1 Topic

Learning objectives

🎯 At the end of this chapter, you will be able to:

- Distinguish between a GET and a POST request and know when each is used.
- Inspect the "Application" tab in Chrome DevTools to find and delete cookies or tokens (essential for testing Login/Logout flows).
- You create your first independent theme: a Login page perfectly structured for automation.
- You use the official QualiAdept validation platform to verify your work and unlock your progress.

4.1 HTTP Methods: GET vs. POST (What happens to the form data?)

In Chapter 3, we built a <form> for adding tasks. But when the user clicks "Save Task", how does that data get to the server? Using HTTP methods. The two most common methods are:

- **GET (Request information):** It is the browser's default method. When you writeemag.roin the browser, you do a GET. If you have a Search form that uses GET, the searched data will appear directly in the URL (ex:site.ro/search?q=laptop).
    - *QA rules:*We NEVER send passwords or sensitive data via GET, as they remain visible in the browser history!
- **POST (Send information):** It is used to send "hidden" data in the request body (HTTP Body), not in the URL. Our "Add Task" form or any "Login/Register" form must use POST.
    - *QA rules:* As an Automation Engineer, we will write API tests (in future modules) in which we will simulate exactly these POST requests, sending JSON files (the data) directly to the server, bypassing the graphical interface!

4.2 Browser Memory: Cookies, Local Storage and Session Storage

When you log in to Facebook, close your browser, and reopen it the next day, you're still logged in. How does Facebook know who you are, if HTTP is a "Stateless" (memoryless) protocol?

The answer: The server gives you a "stamp" (a Token or Cookie), and your browser stores it in a drawer.

There are three main drawers that a QA must know how to open:

- **Cookies:** Small pieces of text. They are automatically sent back to the server with every click you make on the site. They are classically used to maintain the login session.
- **Local Storage:** A virtual "hard drive" in your browser, provided to each website. The data here remains even if you shut down your PC, until it is explicitly deleted (manually or by code). Modern applications store "Bearer Tokens" (long security codes) here.
- **Session Storage:** Similar to Local Storage, but with short memory. If you close the tab, the data here is deleted instantly.

4.3 DevTools: Application Tab (QA's Secret Weapon)

The biggest mistake a tester (manual or automated) making when testing Login functionality is clicking the "Logout" button and thinking they've tested everything. "Logout" only deletes the visual cookie. But what happens if you delete the cookie manually from DevTools? Are you still logged in?

To investigate these memories, Chrome offers us the tab**Application**.

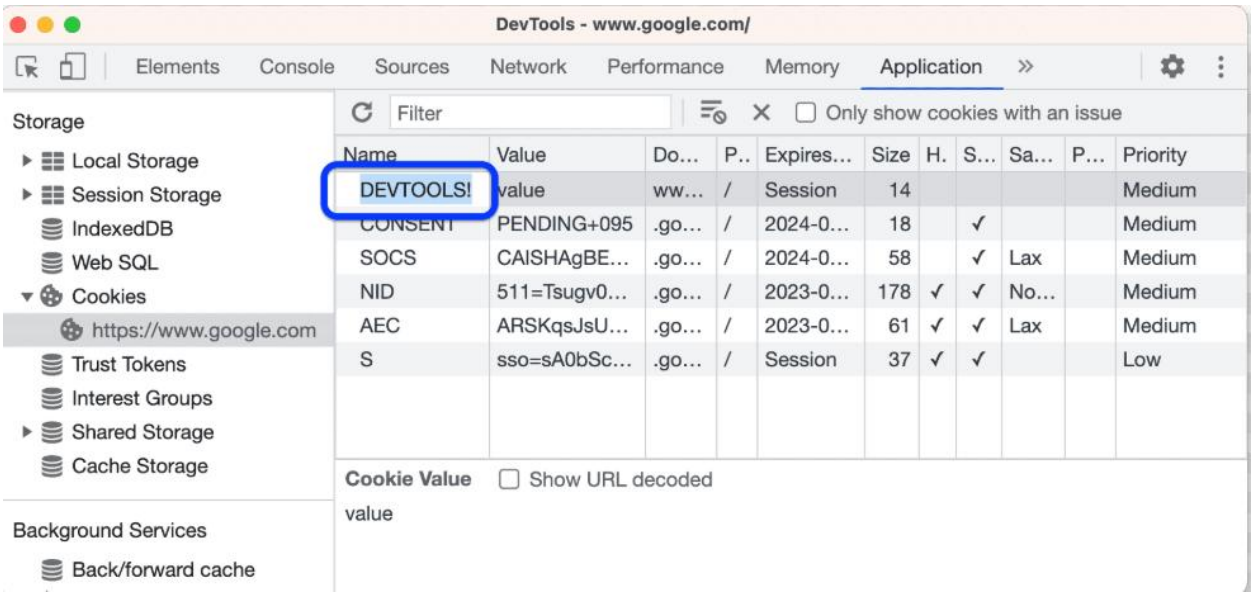

Fig. 4 - The Chrome DevTools panel, with the "Application" tab selected from the top menu, and the "Storage" -> "Local Storage" and "Cookies" subsections highlighted in the left panel

How will the Playwright use this memory?

In the advanced modules, we will learn a brilliant trick: instead of Playwright filling in the username and password for each test (which would take 3 seconds per test), we will have it log in only once, "steal" the Cookie from DevTools using code, and inject it directly into the Browser Memory for the remaining 100 tests. Time saved: enormous!

4.4 Did you know that...?

- **Cookies invented the "Shopping Cart":** The first cookie was created in 1994 by Lou

Montulli (an engineer at Netscape) precisely to allow an online store to remember which products you put in your cart from one page to another.

- **Storage limit:** A Cookie file can only store 4KB of data (very little), while Local Storage can store up to 5MB (enough to save an entire book in your browser's memory).

4.5 Applied Story: "The Postcard and the Armored Parcel"

🧠 The simplest analogy for HTTP GET and POST.

- **The GET request is like a Postcard.** You write your message on the back of the postcard and give it to the postman. On the way, the postman, your neighbors, and anyone else handling the postcard (internet servers) can read your message (because it's visible in the URL). It's perfect for asking "The weather in Bucharest," but terrible for sending "My password is 1234."
- **The POST request is like an Armored Parcel.** The destination address is written on the box, but the contents (the Body) are enclosed inside and sealed. No one on the route sees what's inside. When we make a Login form, we always use POST!

4.6 Practical Exercises

Exercise 1: Cookie Hunt

- Open Google Chrome and go to any site where you are logged in (e.g. YouTube, eMAG or even GitHub).
- Press F12 to open DevTools.
- Go to the tab **Application** (if you don't see it, click the >> arrow in the DevTools menu at the top).
- In the left menu, expand the section **Cookies** and click on the site name.
- *Experiment:*Find the line that seems to be related to session or login (it often has words like "sess", "token", "auth"), right-click on it and choose "Delete". Refresh the page. What happened?

4.7 Answers to Questions & Solutions to Exercises

Solution Exercise 1:

If you deleted the correct cookie (the session cookie) and refreshed the page, **you were logged out instantly** from the site, even if you haven't pressed the official "Logout" button. This is how we, as QA engineers, check if the security of an application is correctly linked to that Cookie and if the server denies access to users who don't have that "stamp" in the browser's memory!

4.8 Homework (Session 1 Project) & QualiAdept Cloud Evaluation Platform

It's time to demonstrate what you've learned! Now that you understand the HTML skeleton and the importance of attributes (id, data-testid), you'll build your first independent, well-thought-out page.*from the start* to be tested by robots.

Your task (Business task):

Build a "Login and Register" HTML page for our application*Task Tracker*.

Technical Requirements (Acceptance Criteria):

- The document must have valid Boilerplate structure (<html>, <head>, <body>).
- The page title (in the browser tab) should be: Task Tracker Login.
- You must have a <form> that contains:
  - An <input> for Email, having id="login-email" and data-testid="input-email".
  - An <input> for Password (type="password"), with id="login-password" and data-testid="input-password".
  - A <button> for submission, with the text "Log In" and the attribute data-testid="btn-submit-login".
- The form must have a Parent container .

Introduction to the Certify QualiAdept Platform

To validate your code and unlock future modules, we'll use real-world engineering tools. We built for you **QualiAdept Cloud Evaluation Platform** (certify.qualiadept.eu).

This platform is your learning ecosystem and works exactly like continuous integration (CI/CD) systems in IT companies:

- 📌 **Automatic Validation Engine (Static Inspection):**The moment you submit your code,

the platform instantly runs a set of tests over your HTML structure. The system deterministically checks for the presence of every attribute, tag, and ID required for future automation.

- 🧭 **Sequential Progression (Drip Content):** The following modules in the course are locked. The only way to advance in the bootcamp is to achieve "Passed" status on the current topic, solving absolutely all the requirements.
- 🎉 **Verifiable Public Portfolio:** All your successfully completed assignments are centralized on a public profile. At the end, you will get a link that you can put directly on your CV, proving to recruiters that you are a **„QualiAdept Verified QA Engineer”.**

How to Self-Validate (Homework Assignment)

Step 1: Write your HTML code in your favorite editor (e.g. VS Code) and make sure it looks visually correct by double-clicking the .html file in the browser.

Step 2: Access the platform certify.qualiadept.eu and log in quickly and securely with your GitHub account.

Step 3: In the LMS Dashboard, locate **Module 01** and press the button **Open Workspace**.

Step 4: Copy your code from VS Code, paste it into the code area on the platform, and click the Submit button.

Step 5 (Feedback and Validation):

- If you wrote the perfect code and placed the correct anchors for automation, you will receive a score of 100% and the green banner with ☑ **Promoted.** The system will mark the module as completed, and Module 2 will prepare for unlocking!
- If validation fails, don't panic! Analyze the error report returned by the platform, see which attribute or ID you missed, go back to your code editor, correct it, and submit the code again.

Be proud of your success!

Once you have successfully validated the theme, return to the platform Dashboard and click on the button **"Share My Profile"**. Share the link you get on the QualiAdept community Discord group (or even on LinkedIn) to brag about your first technical victory. You're officially on the right track!